Title: In-Pass Draft Truncation for Mixture-of-Experts Speculative Decoding on Apple GPUs

Anya Shankar (anyashan), Aanya Kochhar (aanyak)
URL: https://aanya8910.github.io/418-proposal/

Summary

We will make speculative-decoding verification cheaper for Mixture-of-Experts (MoE) models by deciding during the forward pass, layer by layer, which trailing draft rows are worth the extra expert-weight traffic they cause. We will implement this as a fused, lock-free MoE kernel in Metal on a 32-core M5 Max GPU, comparing three schemes for synchronizing threadgroups across phases, and run memory-consistency litmus tests on the same GPU.

Background

Speculative decoding is nearly free on dense models but not on MoE models, because extra verification rows read extra experts. A small draft model proposes k tokens, and the large target model verifies all of them in one forward pass with one row per token. Decoding is memory-bandwidth-bound, and on a dense model every row reads the same weights, so 8 rows cost about 1.12x one row (measured by another Apple-GPU engine).

Our planned target model is IBM's Granite-3.1-3B-A800M (not yet finalized). Each of its 32 MoE layers routes every token to 8 of 40 experts. Rows usually pick different experts, so the verify pass must read the union of their weights. If 8 rows picked experts independently, they would touch about 33 of the 40 experts per layer instead of 8. Real routing is correlated and attention does not grow the same way, so we will measure the actual verification cost in week 1.

Existing systems fix the number of drafts, or choose it before the pass from offline cost profiles (EVICT, EcoSpec). We instead decide during the pass, from the experts the router actually picked:

for each MoE layer l (of L):

```
logits = router(x[0..k])              # parallel over rows
mask[r] = topk_bitmask(logits[r])     # one bit per expert, per row
union[r] = OR(mask[0..r])             # prefix-OR scan across rows
new[r] = popcount(union[r] & ~union[r-1]) # experts row r adds
k = largest prefix where value(r) >= cost(new[r], L - l)
   # value: draft model's cumulative acceptance probability
tasks = build ragged (expert, row-range) tiles for rows [0..k)
parallel for tile in tasks:           # over threadgroups
   y[rows] += gate * expert_matmul(W4bit[e], x[rows])
```

Rows are only removed from the end, and attention is causal, so the kept rows' outputs are unchanged. The work parallelizes at three levels: the mask/prefix-OR step is a scan across rows, expert tiles are independent work items across threadgroups, and each tile is a small 4-bit matrix multiply across threads.

The Challenge

The cut decision sits on the critical path of a bandwidth-bound kernel, and Metal gives few guarantees for the cross-threadgroup handoff it needs.

Workload

- **Dependencies:** within a layer, router → cut → expert matmuls → combine must run in order, and the MoE layers run serially. The cut is a global decision that every expert tile waits on.
- **Memory access:** arithmetic intensity is low (a few rows per weight byte), so performance is set by bytes of 4-bit weights read. Weight reuse exists only when rows share experts, which is exactly the quantity we control.
- **Communication vs. computation:** the producer must publish a small task table to all threadgroups each layer; this handoff is cheap in bytes but expensive in latency relative to a decode-size matmul.
- **Divergence and imbalance:** experts receive different numbers of rows, so tiles are ragged. Static assignment would leave threadgroups idle, which is why workers claim tiles from a lock-free queue.
- **Timing of the decision:** a cut at layer l only saves the remaining layers after l, so early cuts must be predicted from little routing information. Whether early routing predicts later traffic determines whether in-pass truncation can beat a pre-pass rule.

System constraints (Metal)

- Atomics are relaxed-only; ordering must come from device-scope fences on coherent(device) buffers.
- GPU L1 caches are not coherent across cores, so a published table can be read stale without the right fences.
- There is no guarantee all threadgroups are co-resident, so no threadgroup may wait on one that might not be running. Our design avoids this: workers only spin on a flag set by a threadgroup that is already running.
- Kept rows must be bit-identical to an untruncated pass, so the work queue must not change floating-point accumulation order for a given row.

What we hope to learn: how much a data-dependent in-pass decision costs on a GPU without strong synchronization primitives, and which synchronization scheme makes that handoff cheapest.

Resources

We start from MLX's stock operations rather than from scratch, and write the custom kernel ourselves.

Resource	Use
MacBook with 32-core M5 Max GPU	Primary platform: MLX baseline, fused Metal kernel, litmus tests
MLX and mlx-lm	Model loading, baseline implementation, gather_qmm as the production reference
Granite-3.1-3B-A800M, 4-bit (planned, not yet finalized)	Target model
Granite-3.1-1B-A400M (planned; tokenizer match to confirm)	Draft model

References

- Pan et al., Making Every Verified Token Count: Adaptive Verification for MoE Speculative Decoding (EVICT), 2026.
- Apple, Metal Shading Language Specification (memory coherency, coherent(device), atomic_thread_fence with thread scopes).

Optional: PSC Bridges-2 (H100 or V100) only if we add the CUDA mirror kernel.

Goals and Deliverables

Success means a correct, fast fused Metal MoE kernel with in-pass truncation and a measured comparison of three synchronization schemes.

Plan to achieve

1. A cost model of verification cost vs. experts read, measured on the M5 Max for draft lengths 1–8.
2. An MLX baseline of in-pass truncation using stock operations, plus two comparison policies: fixed draft length and an EVICT-style pre-pass rule.
3. A fused Metal kernel that runs one MoE layer in one dispatch: CAS-elected producer, task table published through coherent buffers, device-scope fences and an atomic ready flag, and workers claiming ragged expert tiles from a lock-free queue.
4. Three synchronization variants of that kernel: global barriers, per-expert completion counters, and separate dispatches, with measured synchronization overhead per layer.
5. Correctness: kept rows bit-identical to an untruncated pass, checked on recorded routing traces.
6. Evaluation: tokens/s, MoE time per round, experts read per round, synchronization overhead, and fraction of peak bandwidth relative to MLX's gather_qmm. Target: the fused kernel within [X]% of gather_qmm bandwidth on the same expert set, because both are bound by the same 4-bit weight reads and ours adds only the handoff.

Hope to achieve

1. Memory-consistency litmus tests (e.g., message passing, store buffering) on the M5 Max GPU, with and without fences, to check which reorderings Metal actually exhibits.
2. In-pass truncation beating the pre-pass rule end to end in tokens/s.
3. Optional, only if far ahead: a mirrored CUDA kernel on Bridges-2 for a Metal-vs-CUDA synchronization comparison.

If work goes slowly (week-1 go/no-go)

In week 1 we measure how much expert usage varies between rounds and whether early layers' routing predicts later layers' traffic. If not, in-pass truncation cannot beat a pre-pass rule, and we deliver the cost model, the pre-pass rule, and the Metal kernel as a synchronization and memory-consistency study.

Analysis questions

- How many experts does in-pass truncation save per round compared with fixed and pre-pass policies at equal accepted tokens?
- What does each synchronization scheme cost per layer on Metal, and where does the time go?
- Which reorderings does the M5 Max GPU actually exhibit in litmus tests, and does that match Metal's documented memory model?

Demo (optional): a side-by-side run of fixed-length and in-pass verification on the M5 Max showing tokens/s and experts read per round live.

Platform Choice

The M5 Max fits the workload, and Metal's weak guarantees are what make the synchronization study worth doing.

- **Fit for the workload:** decoding is bandwidth-bound, and a 4-bit MoE model fits entirely in the M5 Max's unified memory, so the whole model runs on one GPU with no host-device copies.
- **MLX:** it provides a working model and production MoE kernel (gather_qmm) to build on and compare against, so our effort goes into the kernel rather than model plumbing.

- **Metal as a learning target:** relaxed-only atomics, non-coherent L1 caches, and no co-residency guarantee force us to design the producer/worker handoff carefully, and make the synchronization comparison meaningful.

Schedule

The go/no-go decision lands at the end of week 1, a correct first Metal kernel is due by the milestone report (Nov 23), and the final report is due Dec 10 before the poster session on Dec 11.

Week	Plan
Nov 9–15	Proposal (due Nov 9). Record routing traces from the target model with a draft model in MLX. Measure expert-usage variation between rounds and whether early-layer routing predicts later traffic. **Go/no-go decision.**
Nov 16–22	Cost model (verify cost vs. experts read). MLX baseline of in-pass truncation, fixed-length policy, and pre-pass rule. Bit-identity test harness. First Metal kernel using separate dispatches per phase.
Nov 23	**Milestone report due.** Go/no-go result, cost model, baseline numbers, and a correct separate-dispatch Metal kernel.
Nov 23–29 (Thanksgiving, lighter week)	Fused single-dispatch kernel: CAS producer election, task table, fences and ready flag; lock-free tile queue.
Nov 30–Dec 6	Global-barrier and per-expert-counter variants. Evaluation vs. gather_qmm: tokens/s, MoE time per round, experts read, synchronization overhead. Metal litmus tests.
Dec 7–11	Finish evaluation. **Final report due Dec 10. Poster session Dec 11, 5:30–8:30pm.**

-